Vault and secrets management

Manage secrets with HashiCorp Vault: seal and unseal, auth methods, policies, KV, dynamic secrets, PKI, transit encryption, Kubernetes integration, high availability and incident response.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What Vault is and the secrets problem (12 min)
2. Starting Vault: seal, unseal and storage (12 min)
3. Authentication methods and tokens (12 min)
4. Policies and least privilege access (12 min)
5. The KV secrets engine and secret versioning (12 min)
6. Dynamic secrets for databases and clouds (13 min)
7. PKI: short-lived certificates with Vault (13 min)
8. Transit: encryption as a service (12 min)
9. Vault on Kubernetes: injector, CSI and operators (13 min)
10. Vault high availability, Raft and disaster recovery (13 min)
11. Audit logging, monitoring and hardening Vault (13 min)
12. Incident response: rotating and revoking secrets (13 min)
Cheat sheet

Lesson 1. What Vault is and the secrets problem

12 minutes. Understand why static secrets in files and variables are risky and how Vault centralises secrets, access control, leases and audit.

Secrets sprawl is one of the commonest security weaknesses. The goal of a secrets manager is simple: no long-lived secret in a place a person or a repository can copy, and a record of every access.

What you will learn

- The problems with static secrets
- Core Vault ideas: engines, auth, policies and leases
- What Vault can and cannot protect
- Vault and its alternatives

The problem

A static secret has four weaknesses: it is copied (files, chat, wikis), it is shared (many people and services use one password), it is long-lived (rarely rotated), and it is untraceable (nobody knows who used it). Vault attacks each: one store, per-identity access, short leases, and an audit trail.

Core concepts

Concept	Meaning
Secrets engine	A plugin that stores or generates secrets (KV, database, PKI, transit)
Auth method	How a caller proves who it is (token, OIDC, Kubernetes, AppRole)
Policy	Rules that say which paths a caller may read or write
Token	The credential Vault gives after login, with a time to live
Lease	A time limit on a secret; Vault revokes it when it expires
Audit device	A log of every request and response (with secrets hashed)

Diagram: A request to Vault

Part	What it does	What goes wrong
Application	Needs a secret to do its job.	Keeping the secret in a file defeats the point.
Auth method	Verifies the caller's identity and issues a token.	A weak login becomes the new weak point.
Policy check	Allows or denies the requested path.	Wide policies give far too much access.
Secrets engine	Returns or generates the secret, with a lease.	An engine misconfigured with a root account is risky.
Audit log	Records the request, the caller and the result.	If it cannot write, Vault stops answering.

Flow: Application to Auth method / Auth method to Policy check / Policy check to Secrets engine / Secrets engine to Application / Policy check to Audit log

What it protects

Vault limits who can read a secret and for how long. It cannot stop an application that has a legitimate secret from misusing it or logging it. Combine it with least privilege, network controls, and short lifetimes.

MARGA by EknathaLabs marga.eknathalabs.com

Alternatives

Cloud secret managers (AWS Secrets Manager, Azure Key Vault, Google Secret Manager) are simpler when you live in one cloud. Vault shines across clouds and on premises, and with dynamic secrets and PKI. OpenBao is an open source fork of Vault. Kubernetes Secrets alone are only base64 encoded unless you add encryption at rest.

Incident: The password in six repositories

Incident: *A departed contractor still knew a live password.*

```
$ git grep -l "DB_PASSWORD=" -- '*.env' '*.yaml'
billing/.env
reports/config.yaml
ops/docker-compose.yaml
... 6 matches in 4 repositories
```

The same database password had been copied into config files over three years. Rotating it meant finding every copy. The team moved to Vault dynamic credentials, so each service received its own short-lived login.

Common mistakes

- Treating Vault as a nicer place to keep the same shared password
- Giving every application the same broad policy
- Skipping the audit device
- Storing the Vault root token in a file

Check yourself

What is a lease?

Answer: A time limit on a secret

What decides what a token may do?

Answer: Policies

What does an audit device record?

Answer: Every request to Vault

What is a secrets engine?

Answer: A plugin that stores or creates secrets

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Starting Vault: seal, unseal and storage

12 minutes. Learn how Vault encrypts its storage, why it starts sealed, how Shamir key shares and auto-unseal work, and which storage backend to choose.

A running Vault has two states that matter: sealed and unsealed. Understanding them explains most startup problems and most security design.

What you will learn

- The encryption barrier and the seal
- Shamir shares and auto-unseal
- Choosing a storage backend
- Initialising and protecting the root token

The barrier and the seal

Vault writes data to storage through an **encryption barrier**. Everything past the barrier is encrypted with a data encryption key. That key is encrypted by the root key, and the root key is protected by the **seal**. When Vault is sealed it knows where its data is but cannot decrypt it. Unsealing gives it the root key in memory.

Unsealing

By default, initialisation splits the root key into key shares with Shamir's algorithm. A threshold of shares, for example 3 of 5, is needed to unseal. Give shares to different people.

```
vault operator init -key-shares=5 -key-threshold=3
vault operator unseal
vault status
```

Auto-unseal delegates this to a trusted external system: a cloud KMS, an HSM, or another Vault's transit engine. Vault unseals itself at start without a person. With auto-unseal you receive recovery keys, which are used for sensitive operations rather than unsealing.

Diagram: How Vault unseals

Part	What it does	What goes wrong
Encrypted storage	Holds all data encrypted by the barrier.	Stolen storage alone is unreadable.
Root key	Protects the data key; kept only in memory when unsealed.	Never written to disk in clear.
Shamir key shares	A threshold of shares rebuilds the root key.	Lost shares below threshold mean a permanent seal.
Cloud KMS or HSM	Auto-unseal: the external key wraps the root key.	If the KMS is unreachable after a restart Vault stays sealed.
Unsealed Vault	Serves requests with the keys in memory.	A memory dump of an unsealed node exposes keys.

Flow: Encrypted storage to Root key / Shamir key shares to Root key / Cloud KMS or HSM to Root key / Root key to Unsealed Vault

Storage backends

Integrated storage (Raft) keeps data on each node's disk and replicates among nodes. It needs no extra system and is the recommended choice. Older setups used Consul. Cloud backends exist but give fewer features. Use fast local disks, and back up with Raft snapshots.

MARGA by EknathaLabs marga.eknathalabs.com

Initial setup

`vault operator init` prints the first root token once. Use it only to configure authentication and policies, then revoke it. Day-to-day work should use named identities. Dev mode (`vault server -dev`) is in memory, auto-unsealed, with a known token: great for learning, never for real secrets.

Incident: Sealed all night

Incident: *A node restart left Vault unreachable until morning.*

```
$ vault status
Key                Value
Seal Type          shamir
Initialized        true
Sealed             true
Unseal Progress    1/3
```

After a kernel patch rebooted the nodes, Vault started sealed. Only one of the five key holders was reachable. The team configured auto-unseal with a cloud KMS key and tested a restart during working hours.

Common mistakes

- Running dev mode anywhere real
- Giving all key shares to one person
- Leaving the initial root token active
- Not testing a restart

Check yourself

Why does Vault start sealed?

Answer: It cannot decrypt its data without the root key

What does auto-unseal use?

Answer: A KMS or HSM

Which backend is recommended?

Answer: Integrated storage (Raft)

What should happen to the initial root token?

Answer: Be revoked after setup

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Authentication methods and tokens

12 minutes. Compare Vault auth methods for people and machines, understand token types, TTLs and renewal, and choose methods that avoid long-lived credentials.

Every Vault request needs a token. How a caller gets one decides how strong the whole system is. The ideal is a login that does not require a secret you must store.

What you will learn

- Methods for people and for machines
- Tokens, TTLs and renewal
- AppRole and the secret-zero problem
- Identity based methods

Methods

Method	For	How it proves identity
OIDC / LDAP	People	Single sign-on through the company identity provider
Kubernetes	Pods	The pod's service account token
AWS / Azure / GCP	Cloud workloads	The cloud's signed identity
AppRole	Pipelines and other machines	A role ID plus a secret ID
Token / userpass	Bootstrap and tests	A direct credential

Enable and use a method:

```
vault auth enable oidc
vault auth enable kubernetes
vault auth list
vault login -method=oidc
```

Tokens

A login returns a **token** with policies and a **TTL**. A service token can be renewed until its max TTL. A periodic token renews forever as long as it renews in time. A batch token is lightweight with no storage. Child tokens are revoked when their parent is, unless created as orphans. Set short TTLs so a leaked token expires quickly.

Diagram: From login to secret

Part	What it does	What goes wrong
Person or workload	Presents proof of identity.	A stolen proof lets someone log in.
Auth method	Checks the proof against the identity source.	A loose binding accepts the wrong caller.
Token with policies and TTL	Short lived and renewable up to a maximum.	Long TTLs turn a leak into a long breach.
Identity provider	OIDC, Kubernetes or cloud IAM.	Its own security sets the ceiling.
Request to a secrets engine	The token is sent with each call.	Tokens in logs are credentials.

Flow: Person or workload to Auth method / Auth method to Identity provider / Auth method to Token with policies and TTL / Token with policies and TTL to Request to a secrets engine

MARGA by EknathaLabs marga.eknathalabs.com

AppRole and secret zero

AppRole splits login into a role ID (not secret, lives with the app) and a secret ID (delivered by the pipeline, single use, short lived). That reduces risk but still needs a trusted way to deliver the secret ID. Platform identity methods avoid this "secret zero" problem entirely.

Binding

Bind roles tightly: a Kubernetes role should name the exact service account and namespace, and a cloud role should name the exact IAM role, not a wildcard.

Incident: Any pod could log in

Incident: *A test pod read production secrets.*

```
$ vault read auth/kubernetes/role/app
Key                                 Value
bound_service_account_names         [*]
bound_service_account_namespaces    [*]
policies                            [prod-db]
```

The role accepted every service account in every namespace, so a pod in a test namespace could obtain the production policy. The role was bound to one service account in the production namespace.

Common mistakes

- Wildcard bindings on Kubernetes or cloud roles
- Very long token TTLs
- Shared AppRole secret IDs
- Using the root token for applications

Check yourself

Which method suits a pod?

Answer: Kubernetes auth

What does a TTL do?

Answer: Limits how long a token is valid

What is secret zero?

Answer: The first secret needed to get other secrets

What should a role binding name?

Answer: The exact service account and namespace

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Policies and least privilege access

12 minutes. Write path-based Vault policies, understand capabilities and the KV version 2 paths, use templated policies for self-service, and test what a token can do.

Authentication says who you are. Policies say what you may do. Most real Vault incidents come from policies that are too broad, not from broken cryptography.

What you will learn

- Path based policies and capabilities
- KV version 2 data and metadata paths
- Templated policies
- Testing and reviewing policies

A policy

```
path "secret/data/billing/*" {
  capabilities = ["read"]
}
path "secret/metadata/billing/*" {
  capabilities = ["list"]
}
path "database/creds/billing-ro" {
  capabilities = ["read"]
}
```

Capabilities: `create`, `read`, `update`, `delete`, `list`, `sudo` (for protected paths) and `deny` (overrides everything). A path ending in `*` matches any suffix. If no policy mentions a path, access is denied.

KV v2 paths

With KV version 2, the API path differs from the CLI path. `vault kv get secret/billing/db` reads `secret/data/billing/db` underneath. Listing uses `secret/metadata/...`. A policy that names the CLI path without `data/` silently gives no access, a common first-day mistake.

Diagram: Policy decision for one request

Part	What it does	What goes wrong
Request path + token	A read of secret/data/billing/db.	The token's policies are all that count.
Policy billing-read	Allows read under secret/data/billing/*.	Too wide a glob adds access by accident.
Policy default	Basic self-service paths.	The default policy applies to every token.
Allow or deny	Union of capabilities, with deny winning.	No match means denied.
Audit log	Records the decision and the caller.	Review denied requests for misconfigurations.

Flow: Request path + token to Policy billing-read / Request path + token to Policy default / Policy billing-read to Allow or deny / Policy default to Allow or deny / Allow or deny to Audit log

Templated policies

```
path "secret/data/{{identity.entity.aliases.auth_kubernetes_abc.metadata.service_account_namespace \
    }}/*" {
  capabilities = ["read"]
}
```

Templating pulls values from the caller's identity, so one policy gives each namespace its own area without a separate policy per team.

MARGA by EknathaLabs marga.eknathalabs.com

Testing

```
vault policy write billing-read billing-read.hcl
vault token create -policy=billing-read -ttl=15m
VAULT_TOKEN=<new token> vault kv get secret/billing/db
vault token capabilities secret/data/payments/db
```

Create a short-lived token with only the new policy and try allowed and forbidden paths. Review policies in version control like code.

Incident: Read on everything

Incident: *A breached service read other teams' secrets.*

```
$ vault policy read app
path "secret/*" {
  capabilities = ["read", "list"]
}
$ vault token capabilities secret/data/payments/db
list, read
```

The shared policy was written in a hurry and attached to every service. When one service was compromised, the attacker could read all paths. The team created one policy per application, limited to its own prefix.

Common mistakes

- One broad policy for every application
- Forgetting the data prefix in KV version 2
- Granting sudo or write where read is enough
- Not putting policies in version control

Check yourself

What does deny do?

Answer: Overrides other capabilities

What path does vault kv get secret/app read in KV v2?

Answer: secret/data/app

What happens if no policy matches a path?

Answer: Access is denied

Why use templated policies?

Answer: One policy can serve many teams

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. The KV secrets engine and secret versioning

12 minutes. Store static secrets in the KV version 2 engine, use versions and soft deletes, patch single keys, and know when static secrets are the wrong choice.

The KV engine is the simplest and most used. It is a good home for secrets that exist outside Vault, such as third party API keys, and it adds versioning and audit that a file never had.

What you will learn

- Enabling KV and reading and writing secrets
- Versions, soft delete and destroy
- Patch and check-and-set
- Organising paths and avoiding sprawl

Using KV

```
vault secrets enable -path=secret kv-v2
vault kv put secret/billing/stripe api_key=sk_live_example
vault kv get secret/billing/stripe
vault kv get -field=api_key secret/billing/stripe
vault kv patch secret/billing/stripe webhook=whsec_example
```

`put` replaces the whole secret with a new version. `patch` updates only the keys you give and keeps the rest. Always prefer `patch` for partial changes.

Versions

```
vault kv get -version=2 secret/billing/stripe
vault kv metadata get secret/billing/stripe
vault kv delete secret/billing/stripe
vault kv undelete -versions=3 secret/billing/stripe
vault kv destroy -versions=1 secret/billing/stripe
```

`delete` marks a version deleted but keeps it for recovery. `destroy` removes it for good. Set `max_versions` so history does not grow forever.

Diagram: A secret and its versions

Part	What it does	What goes wrong
Write: version 1	First value stored.	Old values may be secret too.
Write: version 2	Changed API key.	Services still reading version 1 keep the old key.
Soft delete	Version marked deleted but recoverable.	Looks gone but still exists.
Destroy	Version data is permanently removed.	Cannot be undone.
Check and set	A write must name the current version.	Without it concurrent writers overwrite each other.

Flow: Write: version 1 to Write: version 2 / Write: version 2 to Soft delete / Soft delete to Destroy / Check and set to Write: version 2

Check-and-set

With `-cas=<current version>`, a write fails if someone else wrote in the meantime. You can require it for a whole mount with `cas_required`.

Paths

Use a consistent layout such as `secret/<team>/<app>/<name>`. Match it to policies, so a team's access is one prefix. Do not store large files or high-rate data in KV. It is for small secrets.

MARGA by EknathaLabs
marga.eknathalabs.com

When not to use it

If the target supports dynamic credentials (databases, cloud IAM), use those instead. A static password in KV is still a long-lived secret, only better protected.

Incident: The overwritten key

Incident: *A deploy failed because a secret key had disappeared.*

```
$ vault kv get -version=6 secret/app
api_key    sk_live_example
webhook    whsec_example
$ vault kv get -version=7 secret/app
api_key    sk_live_example2
```

Version 7 had been written with put and only one key, so the webhook key vanished. The team recovered it from version 6, then moved everyone to patch and enabled cas_required on the mount.

Common mistakes

- Using put when patch was meant
- Never setting max_versions
- Storing large files in KV
- Keeping static database passwords in KV when dynamic ones are possible

Check yourself

What does patch do?

Answer: Updates given keys and keeps the rest

What does destroy do?

Answer: Removes a version permanently

What does check-and-set prevent?

Answer: Overwriting a newer version by mistake

What is KV best for?

Answer: Static secrets that cannot be generated

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Dynamic secrets for databases and clouds

13 minutes. Generate short-lived database and cloud credentials on demand with Vault, understand leases and revocation, and rotate the root credential Vault uses.

Dynamic secrets are Vault's best idea. Nothing to store, nothing to rotate by hand, nothing shared. Each service gets its own login that expires.

What you will learn

- How dynamic secrets work
- Configuring the database engine
- Leases, renewal and revocation
- Cloud credentials and rotating the root user

The idea

The application asks Vault for credentials. Vault creates a user in the database with a random password and a limited set of grants, returns it with a lease, and removes it when the lease expires or is revoked. Each request gets a different user, so the audit log can name the caller.

Configuring a database

```
vault secrets enable database
vault write database/config/orders \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@db.internal:5432/orders" \
  allowed_roles="orders-ro" \
  username="vault_admin" password="change-me"
vault write database/roles/orders-ro \
  db_name=orders \
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL '{{ \
    expiration}}'; GRANT SELECT ON ALL TABLES IN SCHEMA public TO \"{{name}}\";" \
  default_ttl=1h max_ttl=24h
vault read database/creds/orders-ro
```

Diagram: A dynamic database credential

Part	What it does	What goes wrong
Application	Requests credentials at startup or before expiry.	Caching them forever defeats the lease.
Vault database engine	Runs the creation SQL with its admin account.	The admin account is a high value target.
Database	Holds the new temporary user.	Users left behind show a failed revoke.
Lease with TTL	Tracks the credential until it expires.	Renewing beyond max TTL is refused.
Revocation	Drops the user at expiry or on demand.	Revoke by prefix after a breach.

Flow: Application to Vault database engine / Vault database engine to Database / Vault database engine to Lease with TTL / Lease with TTL to Revocation / Revocation to Database

Leases

Each credential has a lease ID. The application renews it before it expires, up to the max TTL, then asks for new ones. `vault lease revoke <id>` deletes one user. `vault lease revoke -prefix database/creds/orders-ro` deletes all of them, a strong response to a suspected leak.

Rotating the root user

Vault needs a privileged database account. Run `vault write -f database/rotate-root/orders` and Vault changes that account's password to one only it knows, so even the person who set it up no longer has it.

MARGA by EknathaLabs
marga.eknathalabs.com

Cloud

The AWS engine can create IAM users or, better, issue temporary STS credentials for a role. Azure and Google engines work similarly. Set short TTLs and narrow roles.

Incident: The credential that died on schedule

Incident: *A leaked connection string was already useless.*

```
$ vault read database/creds/orders-ro
Key                Value
lease_duration     1h
username           v-oidc-orders-ro-1m3k9x-1730871442
password           A1b-example-only
$ psql -U v-oidc-orders-ro-1m3k9x-1730871442 -h db.internal orders
FATAL:  role "v-oidc-orders-ro-1m3k9x-1730871442" does not exist
```

A developer pasted a connection string into a public issue tracker. By the time a scanner found it, the TTL had passed and Vault had dropped the database user. The team only needed to confirm in the audit log that nothing used it.

Common mistakes

- Giving the Vault admin account more rights than it needs
- Long TTLs on dynamic credentials
- Caching credentials past their lease
- Not rotating the root credential

Check yourself

What does Vault create for each request?

Answer: A unique database user with a lease

What does revoking a lease do?

Answer: Deletes the credential

What does rotate-root do?

Answer: Changes the admin password to one only Vault knows

Which AWS credentials are preferred?

Answer: Temporary STS credentials

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. PKI: short-lived certificates with Vault

13 minutes. Run a private certificate authority with the PKI engine, design a root and intermediate hierarchy, issue short-lived certificates from roles, and plan for revocation and renewal.

Mutual TLS between services needs a steady supply of certificates. Doing that by hand does not scale. A private CA that issues short-lived certificates through an API does.

What you will learn

- Root and intermediate CAs
- Roles that limit what can be issued
- Issuing and renewing certificates
- Revocation, CRLs and ACME

Hierarchy

A **root CA** signs only intermediates and should be rarely used, kept offline or in its own protected mount. An **intermediate CA** signs the leaf certificates that services use. If the intermediate is compromised you replace it; the root stays trusted.

```
vault secrets enable -path=pki pki
vault secrets tune -max-lease-ttl=87600h pki
vault write pki/root/generate/internal common_name="Example Root" ttl=87600h
vault secrets enable -path=pki_int pki
vault write pki_int/intermediate/generate/internal common_name="Example Intermediate"
```

You then sign the intermediate's request with the root and set the signed certificate back.

Roles

```
vault write pki_int/roles/internal-svc \
  allowed_domains="svc.example.internal" \
  allow_subdomains=true \
  max_ttl=72h
vault write pki_int/issue/internal-svc common_name=api.svc.example.internal ttl=24h
```

A role says which names a caller may request and the longest lifetime. Policies decide who may use a role.

Diagram: A private CA with Vault

Part	What it does	What goes wrong
Root CA (offline)	Signs the intermediate, then stays out of daily use.	A leaked root key means rebuilding all trust.
Intermediate CA in Vault	Signs service certificates.	Its key is the one used every day.
PKI role	Limits domains, key types and TTL.	A wildcard role can issue for any name.
Service	Requests and renews its certificate.	Missing renewal still causes expiry.
Trust bundle	The root certificate distributed to clients.	Clients that lack it reject the chain.

Flow: Root CA (offline) to Intermediate CA in Vault / Intermediate CA in Vault to PKI role / PKI role to Service / Root CA (offline) to Trust bundle / Service to Trust bundle

Issuing and renewing

Applications ask for a certificate, or a sidecar or agent does it and writes files for the service. Because lifetimes are short, the agent renews at about two thirds of the lifetime. Cert-manager in Kubernetes can use Vault as an issuer.

MARGA by EknathaLabs marga.eknathalabs.com

Revocation

With a 24 hour life, waiting for expiry is often enough. For longer lifetimes Vault keeps a CRL and can serve OCSP. Newer Vault versions support ACME, so standard ACME clients can request certificates from your internal CA.

Incident: The weekend expiry

Incident: *An internal API stopped because its certificate had expired.*

```
$ echo | openssl s_client -connect api.internal:443 2>/dev/null | openssl x509 -noout -enddate
notAfter=Nov  2 00:00:00 2026 GMT
$ date -u
Sat Nov  7 08:10:00 UTC 2026
```

The certificate had a one year lifetime and renewal was a manual calendar task nobody owned. The team moved to Vault issued certificates with a 24 hour TTL, renewed by an agent every 16 hours.

Common mistakes

- Issuing leaf certificates directly from the root
- Roles that allow any domain
- Long lifetimes without a revocation plan
- Not alerting when renewal fails

Check yourself

What should sign leaf certificates?

Answer: An intermediate CA

What does a PKI role limit?

Answer: Domains and TTLs that can be issued

Why use short lifetimes?

Answer: Revocation matters less

What can request certificates with standard clients?

Answer: ACME

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Transit: encryption as a service

12 minutes. Use the transit engine to encrypt and decrypt data without the application ever holding the key, rotate keys, rewrap old data and use data keys for large payloads.

Encryption is easy to call and hard to do well. Key storage, rotation and access control are where teams fail. Transit moves all of that out of the application.

What you will learn

- Encrypting and decrypting through the API
- Key versions, rotation and rewrap
- Data keys for large payloads
- Policies that separate encrypt from decrypt

Basic use

```
vault secrets enable transit
vault write -f transit/keys/orders
PT=$(echo -n "4111 1111 1111 1111" | base64)
vault write transit/encrypt/orders plaintext="$PT"
vault write transit/decrypt/orders ciphertext="vault:v1:abc123example"
```

Plaintext is base64 encoded in and out. The result looks like `vault:v1:...`, where `v1` is the key version that encrypted it. Store the ciphertext in your database. The key never leaves Vault.

Rotation and rewrap

```
vault write -f transit/keys/orders/rotate
vault write transit/rewrap/orders ciphertext="vault:v1:abc123example"
```

Rotating creates a new key version for new encryptions; older versions still decrypt. **Rewrap** converts old ciphertext to the latest version inside Vault without exposing plaintext, so you can retire old key versions with `min_decryption_version`.

Diagram: Encrypting a field with transit

Part	What it does	What goes wrong
Application	Sends plaintext, stores ciphertext.	Logging the plaintext defeats the point.
Vault transit	Holds the key and does the cryptography.	Availability matters, since reads need decrypt.
Database	Contains only ciphertext.	A stolen dump alone is useless.
Key rotation	New versions for new data; rewrap for old data.	Skipping rewrap leaves old keys in use.
Separate policies	Services that write need encrypt only.	One policy for both widens exposure.

Flow: Application to Vault transit / Vault transit to Database / Vault transit to Key rotation / Separate policies to Vault transit

Data keys

Sending large files to Vault is slow. Instead request a **data key** from `transit/datakey/plaintext/orders`: you get a plaintext key to encrypt locally and a wrapped copy to store beside the data. Discard the plaintext key after use. To decrypt, ask Vault to unwrap the stored copy.

Policies

Give writers only the `update` capability on `transit/encrypt/orders` and the few services that must read the data `transit/decrypt/orders`. A service that cannot decrypt cannot be tricked into leaking plaintext.

Incident: The dump that was only ciphertext

Incident: *A database backup was exposed.*

MARGA by EknathaLabs marga.eknathalabs.com

```
$ psql -c "select card from payments limit 2"
                      card
-------------------------------------------
 vault:v3:8SDd3WHDOjf7mq69CyCqY...
 vault:v3:Zk0mFQd3pT1xA2lU91nBc...
```

A misconfigured storage bucket exposed a database backup. Because card numbers were encrypted through transit, the backup contained only ciphertext, and the key was never in the dump. The incident was contained without notifying customers.

Common mistakes

- Logging plaintext before encrypting
- One policy for encrypt and decrypt
- Never rotating keys
- Sending very large payloads through the API

Check yourself

What does transit return?

Answer: Ciphertext with a key version

What does rewrap do?

Answer: Moves ciphertext to the latest key without plaintext

Why use data keys?

Answer: To encrypt large data locally

Why split encrypt and decrypt permissions?

Answer: A writer need not read data

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Vault on Kubernetes: injector, CSI and operators

13 minutes. Get Vault secrets into pods using the Vault Agent Injector, the CSI provider and the Vault Secrets Operator, and compare them with External Secrets.

Kubernetes Secrets are only base64 encoded and are easy to over-share. Vault can supply secrets at runtime, with each pod proving its identity.

What you will learn

- Kubernetes authentication
- The Agent Injector sidecar
- CSI provider and Vault Secrets Operator
- Choosing between the patterns

Login

Enable the Kubernetes auth method and create a role that binds a service account and namespace to a policy. The pod presents its service account token, Vault asks the Kubernetes API to validate it, and returns a token with the role's policies.

```
vault auth enable kubernetes
vault write auth/kubernetes/role/orders \
  bound_service_account_names=orders \
  bound_service_account_namespaces=shop \
  policies=orders-read ttl=1h
```

Agent Injector

Annotations on a pod tell a mutating webhook to add a Vault Agent sidecar and init container.

```
metadata:
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "orders"
    vault.hashicorp.com/agent-inject-secret-db: "database/creds/orders-ro"
```

The agent logs in, fetches the secret, writes it to `/vault/secrets/db`, and renews it while the pod runs. The application only reads a file.

Diagram: Three ways to deliver secrets to a pod

Part	What it does	What goes wrong
Pod with service account	Proves identity to Vault.	A shared service account weakens bindings.
Agent Injector sidecar	Writes secrets to a shared volume and renews them.	Adds a sidecar to every pod.
CSI provider	Mounts secrets as a volume at pod start.	Rotation needs the driver to remount.
Vault Secrets Operator	Syncs secrets into Kubernetes Secrets.	The Secret lives in the cluster again.
Vault	Validates the identity and returns secrets.	Unavailable Vault stops new pods.

Flow: Pod with service account to Agent Injector sidecar / Pod with service account to CSI provider / Pod with service account to Vault Secrets Operator / Agent Injector sidecar to Vault / CSI provider to Vault / Vault Secrets Operator to Vault

MARGA by EknathaLabs
marga.eknathalabs.com

Operator and External Secrets

The **Vault Secrets Operator** watches resources such as `VaultStaticSecret` and `VaultDynamicSecret`, and writes the result into a Kubernetes Secret that pods consume the usual way. **External Secrets Operator** offers a similar model for many backends, including Vault and cloud managers.

Choosing

Need a file and rotation without code changes: injector or CSI. Need a native Kubernetes Secret for an existing chart: operator. Remember that any secret copied into a Kubernetes Secret is only as safe as your cluster access control and etcd encryption.

Incident: A pod that stayed Pending

Incident: *A new pod never started because the secret volume failed.*

```
$ kubectl describe pod orders-7f9c | tail -n 3
Warning  FailedMount  2m  kubelet  MountVolume.SetUp failed: permission denied
$ vault read auth/kubernetes/role/orders | grep bound_service_account_names
bound_service_account_names    [order]
```

The role was bound to a service account named order, but the pod used orders. The login was denied and the init container never finished. Correcting the binding fixed the Pending pod.

Common mistakes

- Binding roles to every service account
- Copying all secrets into Kubernetes Secrets
- Skipping etcd encryption when you use synced Secrets
- Using one service account for many apps

Check yourself

What does the Agent Injector add to a pod?

Answer: A Vault Agent sidecar and init container

What does the Vault Secrets Operator create?

Answer: Kubernetes Secrets from Vault

What proves a pod's identity?

Answer: Its service account token

When might you prefer files over Secrets?

Answer: To keep secrets out of etcd

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Vault high availability, Raft and disaster recovery

13 minutes. Run Vault with integrated Raft storage across three or five nodes, understand active and standby nodes, take snapshots, and plan for replication and recovery.

Because everything depends on Vault, its availability sets the availability of every application and pipeline that reads secrets.

What you will learn

- Active and standby nodes
- Running a Raft cluster
- Snapshots and restore
- Replication and disaster recovery options

Active and standby

In a cluster one node is **active** and handles all writes. Standby nodes forward requests to it or redirect clients. If the active node fails, the standbys elect a new leader within seconds. Clients should use a load balancer or an address that follows the active node, with health checks on `/v1/sys/health`.

Raft

```
vault operator raft list-peers
vault operator raft join https://vault-1.internal:8200
vault operator raft snapshot save backup.snap
vault operator raft snapshot restore backup.snap
```

Raft needs a majority to commit: three nodes survive one failure, five survive two. Spread nodes over three zones. Use fast local disks and a low latency network, since each write is replicated before it is acknowledged.

Diagram: A three node Vault cluster

Part	What it does	What goes wrong
Load balancer	Sends clients to a healthy node.	Needs a health check on the active node.
Active node	Handles writes and replicates them.	Losing it triggers an election.
Standby 1	Replicates data and can become active.	Needs the same unseal method.
Standby 2	Replicates data and can become active.	Zone loss with a bad layout loses quorum.
Snapshot storage	Encrypted snapshots in another location.	Untested snapshots may not restore.

Flow: Load balancer to Active node / Active node to Standby 1 / Active node to Standby 2 / Active node to Snapshot storage

Snapshots and restore

Automatic snapshots (Enterprise) or a scheduled job produce files you copy to another place. Snapshots contain encrypted data, so you also need the unseal key or KMS access in the recovery site. Practise a restore into an empty cluster regularly.

Replication

Vault Enterprise offers **performance replication** (secondary clusters serve reads close to users) and **disaster recovery replication** (a warm standby cluster promoted if the primary fails). In the community edition, rely on snapshots and a documented rebuild procedure.

MARGA by EknathaLabs marga.eknathalabs.com

Maintenance

Upgrade standbys first, then step down the active node to move leadership, then upgrade it. Watch `vault.raft.leader.lastContact` and apply latency.

Incident: Lost quorum in a bad layout

Incident: *Both standby nodes were in the same zone as the failed active node.*

```
$ vault operator raft list-peers
Node     Address              State     Voter
vault-1  10.0.1.11:8201       follower  true
vault-2  10.0.1.12:8201       (unreachable)
vault-3  10.0.1.13:8201       (unreachable)
```

All three nodes had been placed in one availability zone to save cross zone traffic cost. When the zone failed, the cluster lost quorum. The team spread the nodes over three zones and tested a zone failure.

Common mistakes

- Putting all nodes in one failure domain
- Taking snapshots but never testing a restore
- Using two nodes
- Upgrading the active node first

Check yourself

How many failures can a five node Raft cluster tolerate?

Answer: Two

What does a standby node do?

Answer: Forwards requests and can take over

What does a Raft snapshot contain?

Answer: Encrypted Vault data

Where should nodes be placed?

Answer: Across failure zones

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Audit logging, monitoring and hardening Vault

13 minutes. Enable audit devices, monitor Vault health and metrics, and apply production hardening: TLS, least privilege, no root tokens, memory and host protection.

A secrets manager is a high value target. The same properties that make it useful, central access to many secrets, make careful logging, monitoring and hardening essential.

What you will learn

- Audit devices and what they record
- Metrics and alerts
- Production hardening checklist
- Operating without root tokens

Audit

```
vault audit enable file file_path=/var/log/vault/audit.log
vault audit enable syslog tag=vault
vault audit list
```

Each request and response is logged as JSON with sensitive values HMAC hashed. You can search for who read a path and when, without storing the secrets in the log. If every audit device fails to write, Vault blocks requests rather than operate without a record, so enable two and monitor disk space. Ship logs to central storage that Vault admins cannot edit.

Monitoring

Enable Prometheus telemetry and alert on: Vault sealed, no leader or frequent leader changes, high request error rates, rising token or lease counts, slow storage (Raft apply time), and certificate expiry of the Vault listener.

Diagram: What to watch and why

Part	What it does	What goes wrong
Sealed state	Vault cannot serve when sealed.	A silent seal looks like a network outage.
Leader and quorum	A stable leader means writes work.	Flapping leaders point to network or disk issues.
Audit device health	If it fails, requests are blocked.	A full disk is the usual cause.
Leases and tokens	Counts show leaks and runaway clients.	Millions of leases slow Vault.
Hardening	Limits what an attacker gets from a node.	Skipped controls mean a bigger breach.

Flow: Sealed state to Leader and quorum / Leader and quorum to Leases and tokens / Audit device health to Leases and tokens / Leases and tokens to Hardening

Hardening checklist

- TLS everywhere, with a certificate from your CA.
- No root token in daily use; revoke after setup, and generate one only for emergencies.
- Run Vault as a dedicated non-root user, with core dumps and swap disabled, and `mlock` enabled.
- Firewall the API and cluster ports to known clients.
- Auto-unseal with a KMS key that has tight access.
- Separate policies for operators and applications, with `sudo` rare.
- Enable namespaces (Enterprise) or separate mounts per team.

MARGA by EknathaLabs marga.eknathalabs.com

- Keep Vault updated.

Without root tokens

If you need privileged work, create a short-lived token with the required policy, or use `vault operator generate-root` with a quorum of key holders for emergencies. Record who did it and why.

Incident: The audit disk filled

Incident: *Vault stopped answering after a long weekend.*

```
$ vault read sys/health
Error reading sys/health: 500 internal server error
$ df -h /var/log/vault
/dev/sda3  20G  20G  0  100% /var/log/vault
$ journalctl -u vault | tail -n 1
audit: no audit devices are available to write to
```

The audit log grew until the disk was full. With no working audit device Vault refused all requests. The team added a second audit device, log rotation and shipping, and a disk space alert.

Common mistakes

- Only one audit device
- Leaving audit logs on the same disk as storage
- Using root tokens for daily work
- Alerting only on process up, not on sealed state

Check yourself

What happens when all audit devices fail?

Answer: Vault blocks requests

What does Vault do to secrets in audit logs?

Answer: HMAC hashes them

Which alert is essential?

Answer: Vault sealed

How should a root token be used?

Answer: Rarely, for emergencies

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Incident response: rotating and revoking secrets

13 minutes. Respond to a leaked token or secret with revocation, lease and token cleanup, rotation, and a sealed-Vault emergency procedure, then review how it happened.

Incidents with secrets are time critical. A short, practised sequence is better than clever improvisation: identify, contain, rotate, investigate and learn.

What you will learn

- Identify and revoke a leaked token
- Revoke leases by prefix
- Rotate static secrets and downstream systems
- Emergency options and the review

Contain

You may not know who owns a leaked token. The **accessor** is a safe reference to it: audit logs show accessors, not tokens.

```
vault token lookup -accessor "$ACCESSOR"
vault token revoke -accessor "$ACCESSOR"
vault lease revoke -prefix database/creds/orders-ro
vault lease revoke -prefix aws/creds/deploy
```

Revoking a token also revokes child tokens and the leases they hold, which removes dynamic credentials issued to it. Revoking by prefix is a broad sweep when you do not know which lease leaked.

Rotate

Static secrets in KV that the token could read must be assumed exposed. Replace them at the source (the vendor dashboard, the database), update KV, and restart consumers. Rotate the database root credential if the database engine was reachable. Use `vault write -f transit/keys/<name>/rotate` for transit keys, then rewrap.

Diagram: Response to a leaked token

Part	What it does	What goes wrong
Detect	A scanner, alert or report shows a leaked token.	Slow detection means a wider blast radius.
Revoke token and leases	Stops further use and drops dynamic credentials.	Revoking the wrong token breaks applications.
Read the audit log	Shows which paths the token touched.	Logs show accessors, not token values.
Rotate exposed secrets	Replace static secrets at the source.	Missing a downstream copy leaves the leak open.
Review and fix the cause	Shorten TTLs, narrow policies, add scanning.	Without it the next leak repeats.

Flow: Detect to Revoke token and leases / Revoke token and leases to Read the audit log / Read the audit log to Rotate exposed secrets / Read the audit log to Review and fix the cause

Investigate

Search the audit log for the token accessor and the client IP. List the paths it read. For each static secret that was read, treat it as exposed.

MARGA by EknathaLabs marga.eknathalabs.com

Emergency options

`vault operator seal` makes Vault stop serving all requests at once; it is a last resort because every application loses access, and unsealing needs your key holders or KMS. Rotating the root key (`vault operator rotate`) and re-keying (`vault operator rekey`) change encryption and unseal keys after a suspected compromise of key shares.

After

Write a short review: how the secret leaked, how long it was valid, what it could reach, and what stops it next time. Common fixes: shorter TTLs, narrower policies, secret scanning in repositories, and dynamic secrets instead of static.

Incident: The token in the public repository

Incident: *A token was found in a public commit.*

```
$ vault token lookup -accessor 7c1a3d6e-example
policies     [ci-read]
ttl          718h
$ vault token revoke -accessor 7c1a3d6e-example
Success! Revoked token (if it existed)
```

The token had a thirty day TTL and the policy ci-read. Revoking it and its leases took two minutes. The audit log showed one read of a test path, so the team rotated one static secret and added secret scanning to the repository.

Common mistakes

- Revoking blindly without knowing what depends on the token
- Rotating Vault but not the downstream systems
- Sealing Vault first and asking questions later
- Skipping the review

Check yourself

What is a token accessor?

Answer: A safe reference to a token

What does revoking by prefix do?

Answer: Revokes all leases under a path

Why rotate static secrets after a leak?

Answer: The token may have read them

When is sealing Vault appropriate?

Answer: As a last resort

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Vault

Seal and unseal, auth, policies, KV, dynamic secrets, PKI, transit and an incident response sequence.

Start and status

```
vault operator init -key-shares=5 -key-threshold=3
vault operator unseal
vault status
vault operator raft list-peers
```

Warning: *Dev mode (`vault server -dev`) is in memory with a known token. Never use it for real secrets.*

Auth

```
vault auth enable kubernetes
vault auth list
vault login -method=oidc
vault token lookup
vault token create -policy=app-read -ttl=15m
```

Warning: *Bind Kubernetes and cloud roles to an exact service account or role. Wildcards let any pod log in.*

Policy

```
path "secret/data/app/*" {
  capabilities = ["read"]
}
path "secret/metadata/app/*" {
  capabilities = ["list"]
}
```

Warning: *KV version 2 reads `secret/data/...`. A policy written without `data/` gives no access.*

KV

```
vault kv put secret/app/db user=app pass=example
vault kv patch secret/app/db pass=newexample
vault kv get -version=2 secret/app/db
vault kv destroy -versions=1 secret/app/db
```

Danger: `put` *replaces the whole secret. Use* `patch` *for a partial change.*

Dynamic and transit

```
vault read database/creds/orders-ro
vault lease revoke -prefix database/creds/orders-ro
vault write database/rotate-root/orders
vault write transit/encrypt/orders plaintext="$(echo -n secret | base64)"
```

Incident sequence

Step	Command or action
Identify	`vault token lookup -accessor "$ACCESSOR"`
Contain	`vault token revoke -accessor "$ACCESSOR"`
Sweep	`vault lease revoke -prefix <path>`
Investigate	Search the audit log by accessor
Rotate	Replace static secrets at the source

Danger: `vault operator seal` *stops every application. Use it only as a last resort.*

Quick health sweep

```
vault status
vault operator raft list-peers
vault audit list
vault token lookup
vault list sys/leases/lookup/database/creds/app
```

MARGA by EknathaLabs marga.eknathalabs.com